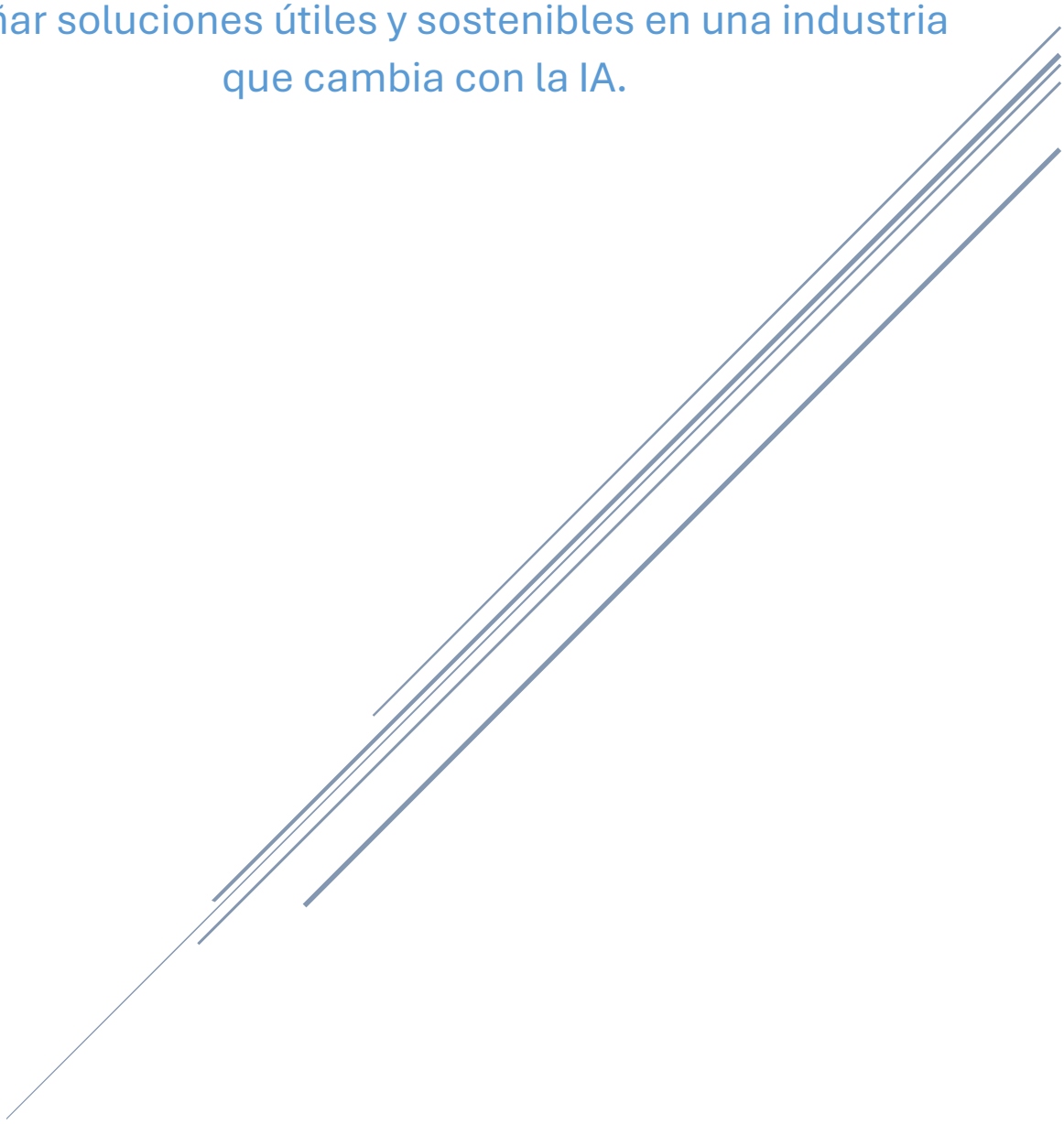
ARQUITECTURA Y DISEÑO DE INTERFACES – COMPENSATORIO 2026

Diseñar soluciones útiles y sostenibles en una industria que cambia con la IA.



IES 9-018 – Tecnicatura Superior en Desarrollo de Software
González Agustina

Introducción

El desarrollo de software evoluciona con los cambios en las arquitecturas, las herramientas y las formas de interacción. Este trabajo analiza tendencias actuales de la industria, su impacto en el diseño y la evolución de las aplicaciones y su relación con los conocimientos adquiridos durante la formación académica.

Para ello, se consultaron materiales del repositorio de la materia y publicaciones de Stripe, Microsoft y NIST. Se abordan tres ejes: las decisiones arquitectónicas y los contratos API; la IA generativa y los agentes autónomos; y la vigencia de los principios estudiados. Finalmente, se aplican estos conceptos al proyecto GanadApp, mi proyecto de desarrollo de software realizado en el marco de la materia y a una propuesta para el sector educativo aplicando conceptos seleccionados de los cuadernillos de la materia y vinculándolos con las tendencias tecnológicas analizadas.

1. Diseñar hoy

Para desarrollar aplicaciones mantenibles y escalables, los equipos deben decidir cómo dividir el sistema y distribuir las responsabilidades entre sus componentes. Entre las alternativas arquitectónicas se encuentran los monolitos modulares, que concentran la aplicación en una unidad de despliegue con responsabilidades separadas, la arquitectura en capas o hexagonal, que organiza la lógica y reduce su dependencia de tecnologías externas; y los microservicios, que distribuyen funcionalidades en servicios independientes. También pueden utilizarse arquitecturas orientadas a eventos para establecer comunicaciones asincrónicas.

La elección depende de la complejidad del dominio, la escala esperada, la autonomía de los equipos y los costos de infraestructura. Los microservicios, por ejemplo, permiten despliegues independientes, pero aumentan la complejidad de comunicación, monitoreo y consistencia de datos, por lo que no siempre son la mejor alternativa.

También es necesario definir contratos claros entre componentes mediante APIs. OpenAPI permite describir operaciones, parámetros, respuestas y estructuras de datos, lo que facilita la documentación, las pruebas y la integración.

Un ejemplo es Stripe. En su artículo de ingeniería *How API changes flow into Stripe's developer products* (2026), Mikael Mello y Anna Mason explican que la empresa adoptó OpenAPI como especificación común para mantener sincronizados sus productos para desarrolladores Anteriormente, utilizaba distintas definiciones y generadores, lo que dificultaba el mantenimiento y la incorporación de funcionalidades. La representación unificada permite generar documentación, SDK y herramientas a partir de las especificaciones de sus APIs. Su repositorio público stripe/openapi contiene estas definiciones y permite generar bibliotecas cliente.

El desarrollo asistido por IA puede acelerar la generación de código, pruebas, documentación y propuestas de arquitectura, pero no garantiza que las soluciones sean seguras, mantenibles o compatibles. Definir requisitos, establecer límites entre componentes y validar operaciones

sigue requiriendo criterio humano. En un proyecto, decisiones como definir los *Non-Goals*, preservar la integridad del historial de movimientos o resolver operaciones contradictorias necesitan evaluación y auditoría. El caso de Stripe demuestra que la automatización aporta valor cuando se apoya en especificaciones comunes y verificables. La IA puede acelerar la implementación, pero la responsabilidad sobre la arquitectura y la calidad continúa siendo del equipo de desarrollo.

2. Tendencias y debate

Uno de los debates actuales de la industria del software es cómo la IA generativa y los agentes autónomos pueden transformar las aplicaciones y la interacción con sus usuarios. A las interfaces gráficas, formularios y menús se suma la posibilidad de expresar instrucciones en lenguaje natural y delegar tareas al sistema.

En el informe *2025: The Year the Frontier Firm Is Born*, Microsoft (2025) afirma: "Where we once said, 'I send emails,' 'I write documents,' 'I create pivot tables,' we'll soon say, 'I create and manage agents'" (apartado «You can buy intelligence on tap»). Esta afirmación anticipa que los usuarios pasarán progresivamente de ejecutar tareas a dirigir agentes que las realicen. Sin embargo, el NIST, en su documento *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile* (2024), advierte que estos sistemas pueden generar información incorrecta o falsa presentada como verdadera y producir riesgos para la privacidad, la seguridad y la confiabilidad de los resultados (apartados 2.2 y 2.4).

Como evidencia, Microsoft presenta encuestas sobre las expectativas de incorporación de agentes, mientras que el NIST proporciona un análisis técnico de los riesgos que deben gestionarse para que estas tecnologías puedan utilizarse de manera confiable. Como opinión, Microsoft destaca el potencial transformador de esta tecnología y el NIST prioriza la gestión de sus riesgos. Como predicción, se plantea que las personas delegarán progresivamente más tareas en agentes autónomos, aunque esto no significa que todas las aplicaciones adopten ese modelo.

Esta tendencia afectaría tanto al diseño como a la arquitectura del software. Las interfaces podrían incorporar agentes capaces de operar en distintos servicios, lo que modificaría la navegación y la distribución de responsabilidades entre usuarios y sistemas. Su implementación requeriría integrar modelos de IA, servicios externos, autenticación, autorización, controles de datos y registros de operaciones. También habría que considerar la latencia, los costos de inferencia, la disponibilidad y la dependencia de proveedores externos.

En la experiencia de usuario, estos cambios podrían simplificar tareas, reducir pasos de navegación y automatizar operaciones repetitivas. Sin embargo, también podrían generar problemas ante instrucciones mal interpretadas, información inexacta o acciones no deseadas. Por ello, las interfaces deberían permitir revisar y corregir resultados, confirmar operaciones importantes y mantener alternativas de interacción tradicionales.

Considero que el valor de estas tecnologías debe evaluarse por los problemas que resuelven y no por su novedad: incorporarlas no garantiza una mejora y su uso debe justificarse por su utilidad, confiabilidad, seguridad y costo, manteniendo la supervisión humana.

3. Revisar lo aprendido

El apartado «Del Vibe Coding al Spec-Driven Development», del repositorio ADI-teoria-y-recursos, plantea que el uso de IA debe guiarse por especificaciones, decisiones arquitectónicas documentadas y supervisión humana. Este enfoque sigue vigente frente a agentes capaces de ejecutar tareas complejas: cuanto mayor sea su autonomía, más necesario será definir límites, criterios de aceptación y mecanismos de revisión.

El apartado «Del Vibe Coding al Spec-Driven Development» del cuadernillo ADI-teoria-y-recursos, sigue vigente:se centra principalmente en la generación de código a partir de especificaciones y en la revisión posterior del resultado. La realidad 2025-2026 muestra agentes capaces de explorar repositorios, ejecutar herramientas, modificar múltiples archivos y proponer planes completos. Por ello, el enfoque SDD debería ampliarse explícitamente hacia la supervisión de acciones (permisos, herramientas habilitadas y registro de cambios), no solo hacia la revisión del código generado. Esta actualización no reemplaza los principios del cuadernillo: los vuelve operativos frente a un nivel de autonomía que en 2024 todavía era limitado.

El material podría ampliar sus ejemplos, ya que la IA no solo genera código a partir de instrucciones, sino que también puede operar sobre repositorios y participar en distintas etapas del desarrollo. Por ello, además de revisar el código, es necesario controlar permisos, herramientas disponibles y cambios realizados. Esto amplía las prácticas de SDD, pero no las reemplaza.

La experiencia de Stripe se relaciona con los contenidos sobre diseño del proyecto y arquitectura: los contratos API ayudan a definir componentes y sus relaciones. La IA puede colaborar en su generación y mantenimiento, pero su definición y validación requieren decisiones técnicas fundamentadas.

La perspectiva de Microsoft sobre la incorporación de agentes de IA amplía el análisis de requisitos funcionales y no funcionales: al delegar tareas en agentes, es necesario especificar qué operaciones pueden realizar, con qué permisos y bajo qué condiciones de intervención humana.

Finalmente, el enfoque de NIST sobre gestión de riesgos de IA complementa los criterios de diseño y calidad estudiados, incorporando consideraciones específicas sobre privacidad, seguridad y confiabilidad. Esto exige contemplar errores de generación, exposición de datos y ejecución de acciones no deseadas.

4. Mirar el propio proyecto

Un ejemplo en GanadApp es el circuito de gestión sanitaria y trazabilidad de los animales. Considero que está bien encaminado porque permite identificar las responsabilidades de cada componente. La funcionalidad se distribuye entre los modelos EventoSanitario e HistorialAnimal, los repositorios SanidadRepository y TrazabilidadRepository, los servicios correspondientes y las rutas de la API. También cuenta con páginas específicas, como /sanidad y /trazabilidad, implementadas mediante sus respectivos templates.

Esta separación facilita la localización de modificaciones sin afectar todo el módulo. Por ejemplo, si fuera necesario cambiar la forma de registrar una vacunación, las modificaciones podrían concentrarse en el repositorio y el servicio de sanidad. Del mismo modo, la consulta del historial se resuelve mediante TrazabilidadService, que reúne los eventos sanitarios y los cambios registrados para un animal, los ordena cronológicamente y permite conservar las anulaciones.

La organización responde a una arquitectura modular por capas y a las necesidades de pequeños y medianos productores, al facilitar el registro de vacunaciones y la consulta de antecedentes sin depender de registros dispersos.

Conservaría esta estructura y el registro histórico de los eventos con su anulación lógica, ya que permiten mantener la integridad de la información y ampliar el sistema sin modificar su organización general. Como mejora, incorporaría un módulo de análisis y reportes que integre los datos de las distintas áreas de GanadApp. Esto permitiría generar informes por establecimiento y período, consultar indicadores sanitarios, observar la evolución de la población animal y analizar el consumo de insumos. A diferencia de las funcionalidades actuales, centradas en la gestión operativa, este módulo ofrecería una perspectiva gloal de la actividad mediante indicadores, gráficos y filtros. Su desarrollo requeriría definir métricas, implementar consultas específicas y diseñar las visualizaciones correspondientes, aprovechando los modelos y servicios existentes. Si bien supone un esfuerzo adicional, permitiría aprovechar mejor la información registrada y facilitar la toma de decisiones de los productores.

5. Adaptarse a un rubro

En educación, una necesidad frecuente es reducir las consultas administrativas repetitivas sobre inscripciones, exámenes, horarios y trámites. Una solución posible sería un asistente institucional basado en IA que responda únicamente a partir de fuentes oficiales versionadas. Estaría destinado a estudiantes y personal administrativo. Los estudiantes consultarían en lenguaje natural; el personal actualizaría la información y supervisaría operaciones.

El sistema requeriría un modelo de datos que separe claramente contenidos oficiales (con versión, fecha de vigencia y fuente) de conversaciones y solicitudes. La arquitectura debería aislar el componente de recuperación de información del componente de interacción, establecer permisos diferenciados y controles de acceso, registrar cada consulta y acción, y exigir confirmación humana antes de iniciar cualquier trámite. También necesitaría mecanismos de derivación al personal cuando la confianza de la respuesta sea baja.

Se trata de una solución nueva para una necesidad concreta del sector, no de añadir IA a una funcionalidad existente.

Conclusiones

La investigación muestra que los cambios tecnológicos no vuelven obsoletos los principios de la ingeniería de software: la modularidad, los requisitos claros, las decisiones documentadas y los contratos entre componentes siguen siendo fundamentales para la mantenibilidad. El caso de Stripe evidencia el valor de las especificaciones comunes para facilitar integraciones y reducir duplicaciones.

La IA generativa y los agentes amplían las posibilidades de desarrollo e interacción, pero también introducen riesgos de confiabilidad, seguridad y privacidad. Los aportes de Microsoft y NIST permiten considerar tanto su potencial como la necesidad de establecer controles y supervisión humana.

El análisis de GanadApp permitió reconocer cómo la separación de responsabilidades y la trazabilidad facilitan el mantenimiento y la evolución de funcionalidades. La propuesta educativa, a su vez, mostró cómo aplicar estas tendencias a otro sector. En conclusión, el desafío no consiste en adoptar cada nueva tecnología, sino en evaluar cuándo aporta valor y cómo integrarla para construir sistemas seguros, confiables y adaptables.

Referencias bibliográficas

- Microsoft. (2025, 23 de abril). 2025: The Year the Frontier Firm Is Born. Work Trend Index Annual Report. Apartado «You can buy intelligence on tap». https://www.microsoft.com/en-us/worklab/work-trend-index/2025-the-year-the-frontier-firm-is-born
- Autio, C., Schwartz, R., Dunietz, J., Jain, S., Stanley, M., Tabassi, E., Hall, P. y Roberts, K. (2024, 26 de julio). Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. NIST AI 600-1. National Institute of Standards and Technology. Apartados 2.2 «Confabulation» y 2.4 «Data Privacy». https://doi.org/10.6028/NIST.AI.600-1
- Mello, M. y Mason, A. (2026). How API changes flow into Stripe's developer products. Stripe Engineering Blog. https://stripe.dev/blog/how-api-changes-flow-into-stripes-developer-products
- - Stripe. (s. f.). Stripe OpenAPI Specification. Repositorio oficial de GitHub. https://github.com/stripe/openapi

Material académico de la asignatura

- Alvarez, Paulo (2026). *ADI — Teoría*. Arquitectura y Diseño de Interfaces: teoría, Tecnicatura Superior en Desarrollo de Software, IES 9-018. https://github.com/IES9018/ADI-teoria-y-recursos/blob/main/cuadernillos/adi-teoria.pdf